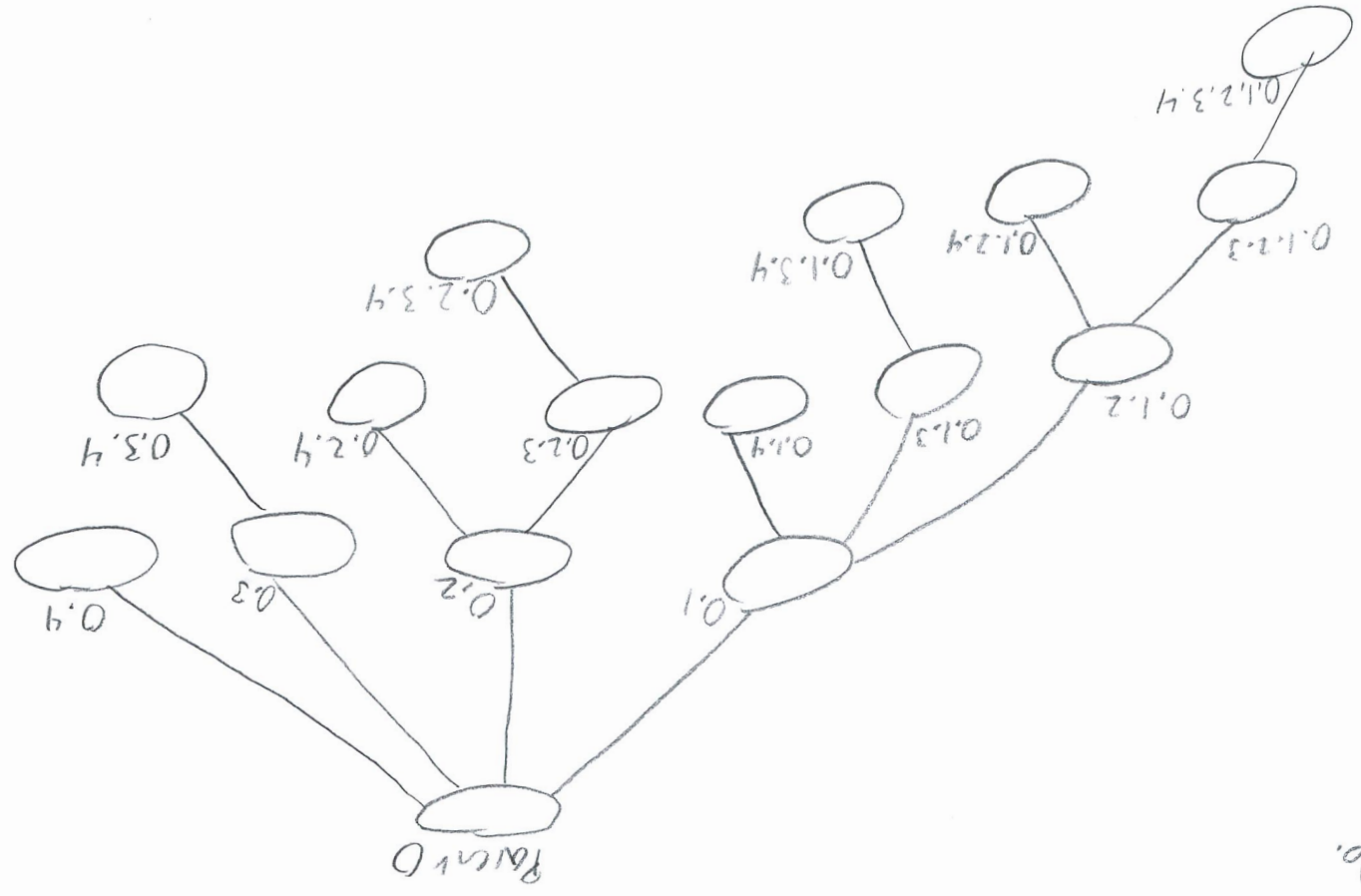


1. a. 16 fork processes



Tyler Davidson

2. a. Then it will save state of the current running process (Registers, Program Counter, Accounting Info) into the process control block (PCB) in memory. Next it will load & fill the values from the PCB back into memory (context switch). After saving the previous context & loading the new one from memory it can now update the register set & begin executing new process.

2.

6.

Single-registry set context switch = 12ns  
 Multi-registry set context switch =  $\begin{cases} 1 \text{ ns} & \text{if next process is in a registry set.} \\ 12 \text{ ns} & \text{if next process is not in a registry set.} \end{cases}$

Let  $P$  = Percent of time the next process is in the registry set.

So  $1-P$  = Percent of time the next process is not in the registry set.

$$M = E(\text{Multi-registry context switch}) = \sum_{x \in X} x \cdot f(x) = P \cdot 1 + (1-P) \cdot 12$$

$$= P + 12 - 12P = 12 - 11P$$

In order for the multi-registry set to have an advantage its average time needs to be less than 10 so.

$$12 - 11P < 10$$

$$2 < 11P$$

$$\frac{2}{11} < P$$

So it needs to be ready 18% of the time.

3.

Let Process A be a process currently running & Process B be the next process we want to context-switch to.

- ① Change from user mode to kernel mode.
- ② Store the context of Process A into its PCB in memory.
- Change the status of A to ready.
- Change the queue pointer of A
- ③ Get the next process (B) from the scheduler & its context from the D3 database.
- Load the context into registers
- Change the status of B to be running
- Remove from ready queue.
- ④ Change from kernel mode back to user mode & resume program execution.

4.

For Line X, the Print Statement Will never occur because the Exec Command Will replace the current running Program with the results of the LS Command.  
For Line Y, the Print Statement will output the string "Parent:" & the value of the integer in nums at index 1 after the children have finished.

5.

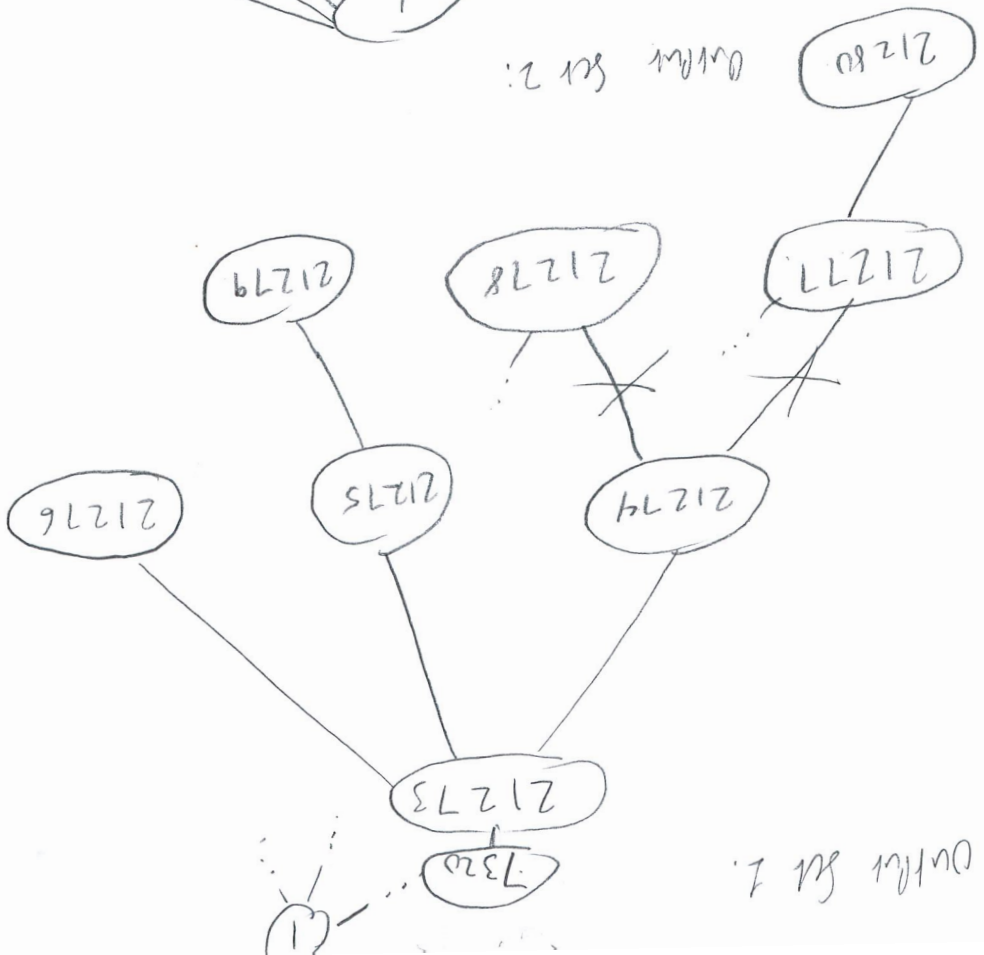
## Synchronous Communication

|                            | System level                                               |                                                       | Programmer level                                         |                                             |
|----------------------------|------------------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------|---------------------------------------------|
|                            | Pros                                                       | Cons                                                  | Pros                                                     | Cons                                        |
| Synchronous Communication  | Simplicity<br>Reliability                                  | Wait time/Latency<br>hard to Parallelize              | Predictable<br>Easier to Debug                           | Managing Wait time "Deadlocks"              |
| Asynchronous Communication | Better Performance/<br>Less Wait time                      | More complex<br>Synchronization                       | Non-blocking operations<br>Can continue other Processing | Race conditions<br>out of order messaging   |
| Automatic Buffering        | Simplifying System<br>handles messages<br>Reduced Blocking | Increased Storage<br>usage. Lack of Control           | Much Easier to Program<br>Less Synchronization issues    | Less Customization<br>Potential bottlenecks |
| Explicit Buffering         | Less System overhead<br>Finer Control                      | Error prone<br>Increased complexity                   | Full control<br>Better understanding                     | more complexity<br>Development time         |
| Send By Copy               | Isolation, Individual<br>Memory values                     | Performance<br>Data Redundancy                        | Avoids side affect<br>Easier                             | memory Address                              |
| Send By Reference          | Fast & Efficient                                           | Risk of Corruption<br>Race conditions/Synchronization | Performance<br>Less Memory Footprint                     | Complexity<br>Harder Debugging              |
| Fixed-Size Messages        | Simpler, Faster                                            | Less flexibility/Wasted Space                         | Predictable, simpler                                     | Wasted Space                                |
| Variable-Size Messages     | Efficient, Flexible                                        | Complex<br>Fragmentation                              | Flexibility, Good<br>for large Data                      | Complexity<br>unpredictable Performance     |

6. Ordinary files are helpful when you want to split up a process & parallelize a task across multiple processes & communicate results from child processes to the parent process but showed them, An example of this would be splitting up a reference calculation in children & sending the results of that calculation back to the parent through the ordinary file.

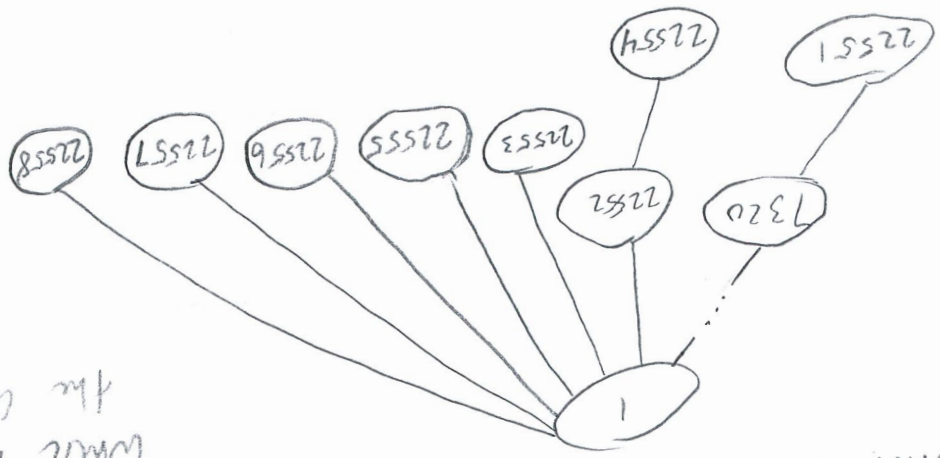
Named files are helpful when you want to communicate between processes that are unrelated in the process tree. An example of this would be communicating logs between different applications to the same process to store & aggregate all logs between the different applications.

I reconstructed what I think the Rouss tree would look like, since some of the PPs were 1, that means that the forest PPs that showed a child finished before he died & exited. Meaning that PPs 21278 & 21277 were orphaned & adopted by the root.



7. Output Set 1.

In this case I cannot reconstruct the tree because there are too many orphaned children where their parents finished & exited before the corresponding children finished.



Output Set 2: